

ADC-aware PTQ and Post-ADC Low-Rank Correction for Llama

Analog in-memory compute · Post-training quantization

Alina Burykina

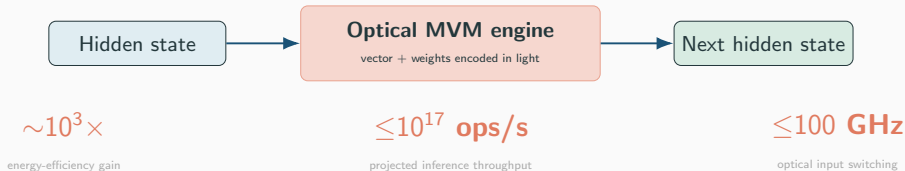
Constructor University — April 2026

Optical GPU for LLM Inference

Photonic tensor core for Transformer linear layers

LLM inference is mostly dense linear algebra

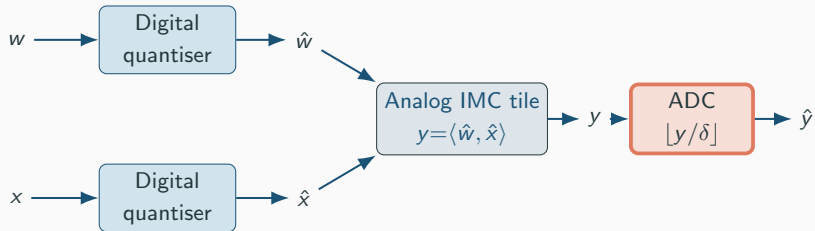
QKV projections · FFN · Output projection



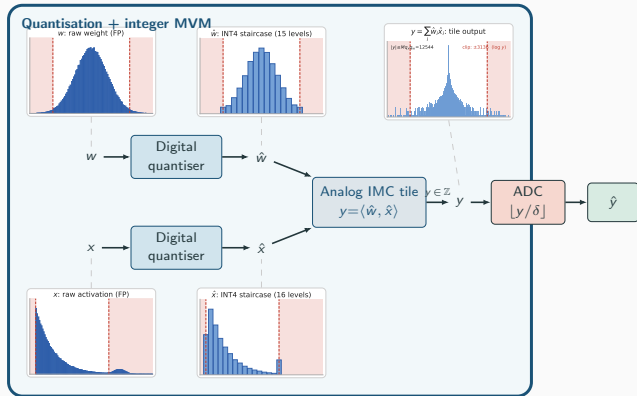
Not a GPU replacement — a specialized photonic accelerator for Transformer linear layers.¹

¹Bandyopadhyay et al., *Single chip photonic deep neural network with accelerated training*, arXiv:2308.05226

Analog MVM Signal Path

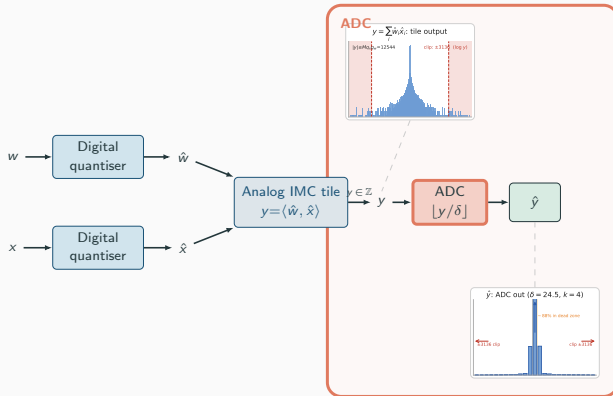


Digital Quantisation



Quantise: $\hat{v} = \text{clip}(\lfloor v/s \rfloor, -q, q)$, $s = \max |v|/q$, $q = 2^{b-1} - 1$ **Dequantise:** $\tilde{v} = s \cdot \hat{v}$

ADC: Reading the Analog Accumulation



ADC read: $\hat{y} = \text{clip}(\lfloor y/\delta \rfloor, -2^{b_a-1}, 2^{b_a-1}-1)$ Step: $\delta = \frac{2M q_x q_w}{2^{b_a} k}$ Dead zone: $|y| < \delta \Rightarrow \hat{y} = 0$

Research Questions

- Q1** Can we **reshape** x, w so that ADC-quantised forward matches FP behaviour *without* retraining the model?
→ *Method I: FlatQuant + trainable diagonals*
- Q2** How low can **pure PTQ** drive PPL_{ADC} on INT8?
→ *block-wise propagated calibration*
- Q3** Does the same recipe work on **INT4 ADC**, where δ is $330\times$ smaller?
→ *staged training, α -mix loss*
- Q4** When PTQ **plateaus**, what is the minimum intervention beyond PTQ?
→ *Method II: Post-ADC residual LoRA*

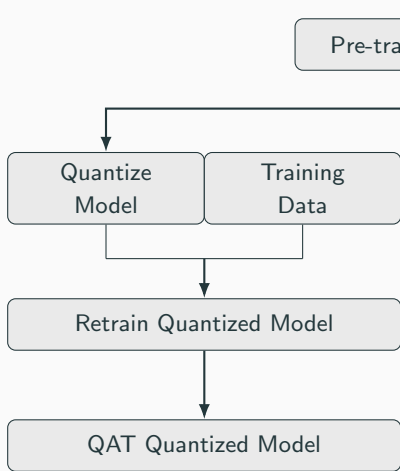
Thesis in one line

*The right place to correct ADC error is **after** the ADC, not before it.*

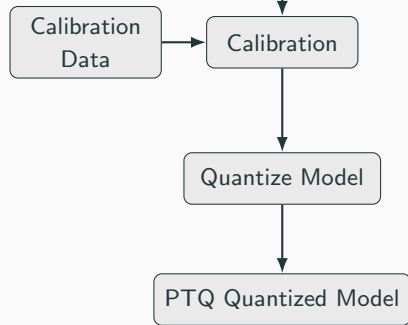
Setup: Llama-3.2-1B, WikiText-2 for calibration, WikiText-2 + C4 for evaluation, ADC spec $b_a=8, k=16, M=256$.

QAT vs PTQ

QAT



PTQ



FlatQuant: Rotate, Flatten, Calibrate per Layer

1. Linear invariance.

For any invertible T :

$$y = xW^T = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^T)}_{\tilde{W}^T}.$$

Choose T so $\tilde{x}$ is *flat* $\Rightarrow$ integer quantisation & ADC coverage improve.

FlatQuant: Rotate, Flatten, Calibrate per Layer

1. Linear invariance.

For any invertible T :

$$y = xW^{\top} = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^{\top})}_{\tilde{W}^{\top}}.$$

Choose T so $\tilde{x}$ is *flat* $\Rightarrow$ integer quantisation & ADC coverage improve.

2. Kronecker factorisation.

Parameterise $T \in \mathbb{R}^{d \times d}$ as

$$T = T_1 \otimes T_2, \quad T_1 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}, \quad T_2 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}$$

params $\mathcal{O}(d)$ instead of $\mathcal{O}(d^2)$.

FlatQuant: Rotate, Flatten, Calibrate per Layer

1. Linear invariance.

For any invertible T :

$$y = xW^\top = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^\top)}_{\tilde{W}^\top}.$$

Choose T so $\tilde{x}$ is *flat* $\Rightarrow$ integer quantisation & ADC coverage improve.

2. Kronecker factorisation.

Parameterise $T \in \mathbb{R}^{d \times d}$ as

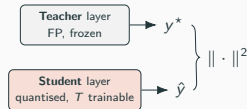
$$T = T_1 \otimes T_2, \quad T_1 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}, \quad T_2 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}$$

params $\mathcal{O}(d)$ instead of $\mathcal{O}(d^2)$.

3. Teacher-student per-layer calibration.

For each linear layer L :

- **Teacher** (FP): $y^* = L_{\text{FP}}(x^*)$
- **Student** (quantised + T): $\hat{y} = \hat{L}_T(\hat{x})$
- Minimise $\|\hat{y} - y^*\|^2$ over T 's only.



Our Extension I: Block-wise Calibration & Propagation

From per-projection to per-block.

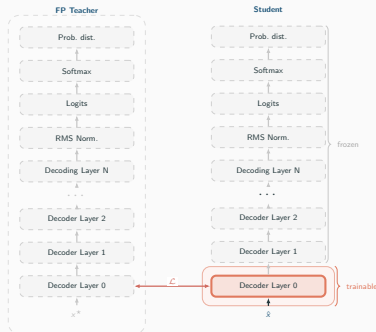
We reconstruct the *whole transformer block*:

$$\mathcal{L} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2$$

Each block is trained *one at a time* — here, Decoder Layer 0.

Propagation.

Input to block ℓ is the ADC-quantised output of block $\ell-1$.



Our Extension I: Block-wise Calibration & Propagation

From per-projection to per-block.

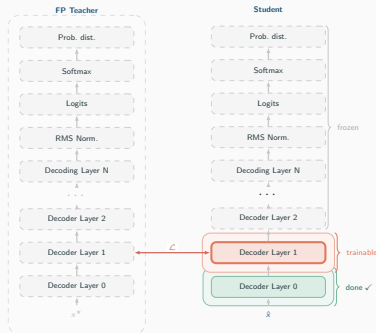
We reconstruct the *whole transformer block*:

$$\mathcal{L} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2$$

Now **Decoder Layer 1**. Its input is the **ADC-quantised output of Layer 0**.

Propagation.

Error accumulates *the same way at train time and at inference*.



Our Extension I: α -Mixed Objective

From per-projection to per-block.

FlatQuant calibrates each linear layer separately (MSE on its output). We reconstruct the *whole transformer block*:

$$\mathcal{L} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2$$

Gradients flow through $q/k/v \rightarrow \text{attn} \rightarrow o \rightarrow \text{gate/up} \rightarrow \text{down}$.

α -mixed objective.

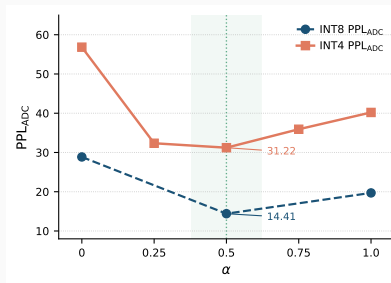
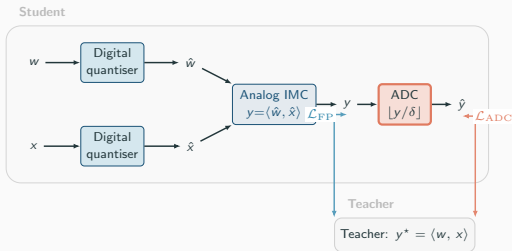
Feed both FP and ADC inputs through the student block in one step:

$$\mathcal{L}(\alpha) = (1-\alpha)\mathcal{L}_{\text{FP}} + \alpha\mathcal{L}_{\text{ADC}}$$

$\alpha=0$: clean — easy to fit, ignores ADC noise.

$\alpha=1$: realistic — matches inference but hard to optimise (noisy grads).

$\alpha=0.5$: *both* signals at once.



Our Extension I: Block-wise Calibration & α -Mix

From per-projection to per-block.

FlatQuant calibrates each linear layer separately (MSE on its output). We reconstruct the *whole transformer block*:

$$\mathcal{L} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2$$

Gradients flow through q/k/v $\rightarrow$ attn $\rightarrow$ o $\rightarrow$ gate/up $\rightarrow$ down, so **ADC error from earlier projections is seen by later ones during optimisation**.

Propagation across blocks.

Input to block ℓ is the ADC-quantised output of block $\ell-1$, *not* its FP teacher activation. The error accumulates *the same way at train time and at inference*.

α -mixed objective.

Feed both FP and ADC inputs through the student block in one step:

$$\mathcal{L}(\alpha) = (1-\alpha)\mathcal{L}_{\text{FP}} + \alpha\mathcal{L}_{\text{ADC}}$$

$\alpha=0$: clean — easy to fit, ignores ADC noise.

$\alpha=1$: realistic — matches inference but hard to optimise (noisy grads).

$\alpha=0.5$: *both* signals at once.

α	INT8 PPL _{ADC}	INT4 PPL _{ADC}
0.0	28.86	38.5
0.5	14.41	27.56
1.0	19.70	31.8

What this buys us (INT8)

PPL_{ADC} 28.86 $\rightarrow$ **14.41** just from switching the loss from per-projection MSE to block α -mix. No extra parameters.

Our Extension II: Trainable Diagonals (add_diag)

Motivation. A rotation T flattens the *shape* of the activation cloud, but not its *per-channel scale*. On INT4 ADC this leaves residual outliers that shove a few channels into the clipping region.

Our Extension II: Trainable Diagonals (add_diag)

Motivation. A rotation T flattens the *shape* of the activation cloud, but not its *per-channel scale*. On INT4 ADC this leaves residual outliers that shove a few channels into the clipping region.

Fix. Absorb a trainable diagonal $D = \text{diag}(d_1, \dots, d_c)$ into the rotation:

$$\tilde{T} = T \cdot D, \quad d_i \in [10^{-4}, 10]$$

Invariance preserved — equal and opposite scaling on the next layer:

$$y = (xD^{-1}T^{-1})(TDW^{\top}).$$

Our Extension II: Trainable Diagonals (add_diag)

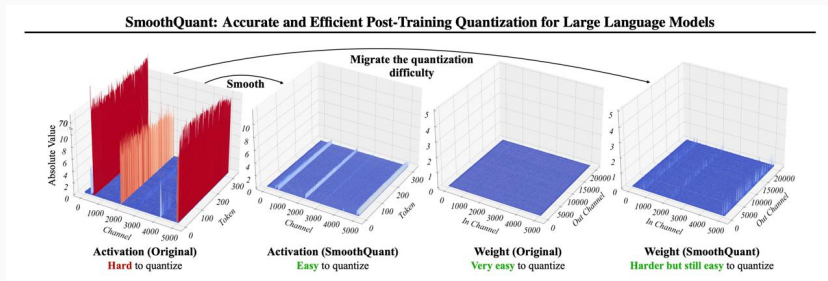
Motivation. A rotation T flattens the *shape* of the activation cloud, but not its *per-channel scale*. On INT4 ADC this leaves residual outliers that shove a few channels into the clipping region.

Fix. Absorb a trainable diagonal $D = \text{diag}(d_1, \dots, d_c)$ into the rotation:

$$\tilde{T} = T \cdot D, \quad d_i \in [10^{-4}, 10]$$

Invariance preserved — equal and opposite scaling on the next layer:

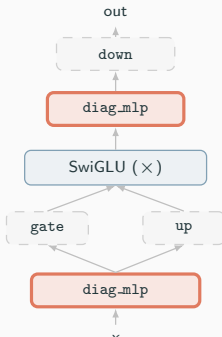
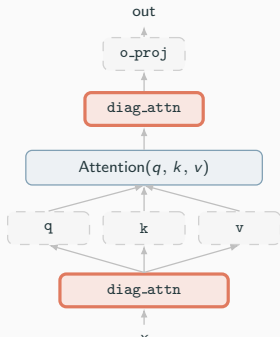
$$y = (\times D^{-1} T^{-1})(TDW^\top).$$



Our Extension II: Trainable Diagonals (add_diag)

Where we place them.

- `diag_attn`: before `q/k/v/o` projections
(absorbs attention-head variance)
- `diag_mlp`: before `gate/up/down` projections
(absorbs FFN outlier channels — SwiGLU)



Ablation (Llama-3.2-1B, w4a4, ADC PPL↓):

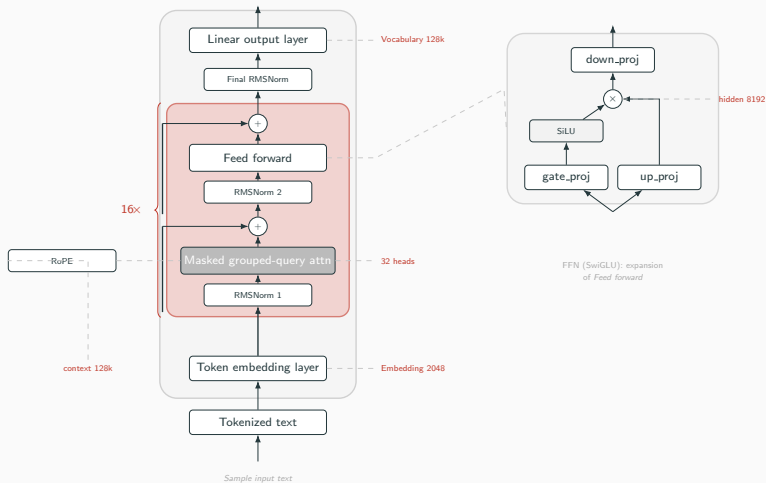
FlatQuant +	PPL _{bypass}	PPL _{ADC}	dead-rate
— (base)	13.17	2355	80.9%
<code>diag_mlp</code> only	12.58	1090	45.2%
<code>diag_attn</code> only	12.89	830	32.1%
both diagonals	12.04	420	10.4%

(numbers to be re-filled from v3 logs)

Why it matters for ADC

Diagonals absorb the per-channel dynamic range — so a single δ can serve every channel. This is the only PTQ knob we have that directly targets the *fixed-step* nature of ADC.

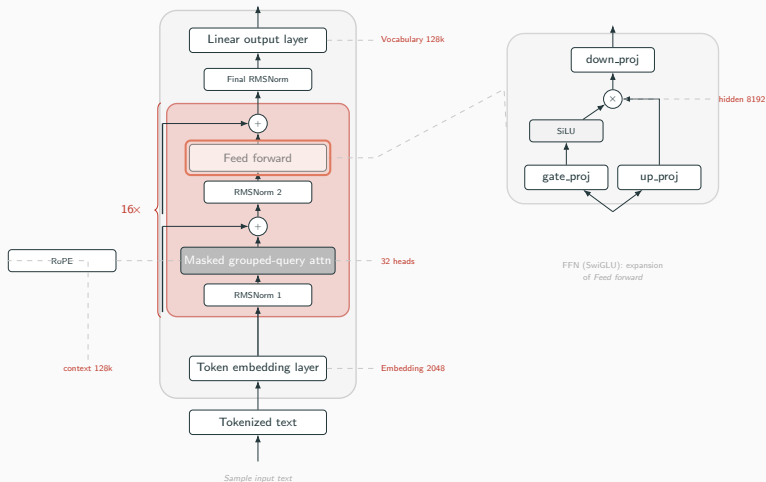
Staged Training: What Is Learnt, When



Per-block params

7 Kron. rotations $T_{q,k,v,o,g,u,d}$;
3 MLP diagonals $D_{g,u}^m$; 3 attn
diagonals $D_{q,k,v}^a$.

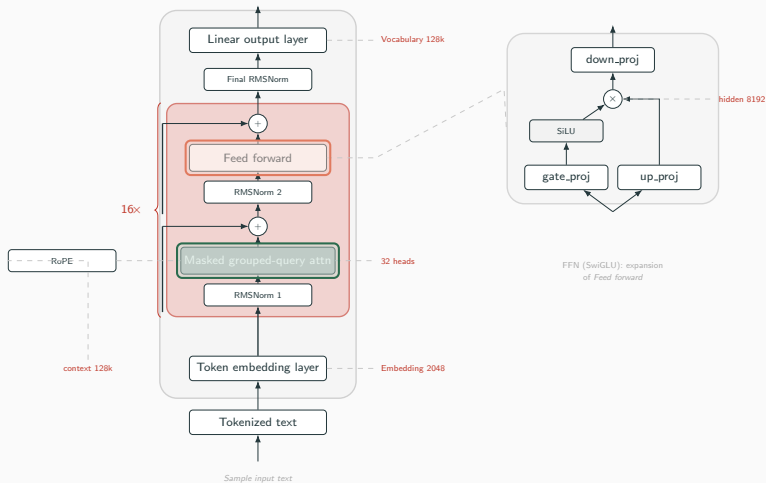
Staged Training: What Is Learnt, When



Stage A | MLP-first

Freeze attention. Train $T_{g,u,d}$ and $D_{g,u}^m$ with $\alpha=0$ — clean forward warmup.

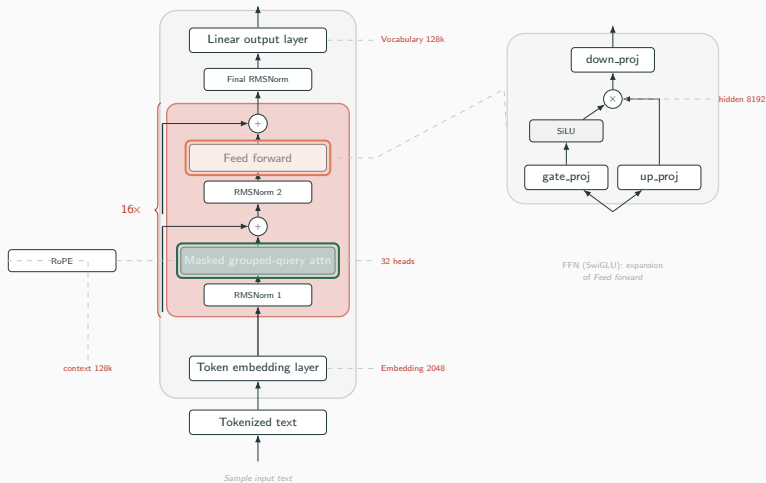
Staged Training: What Is Learnt, When



Stage B | full block

Unfreeze attention. Add $T_{q,k,v,o}$ and $D_{q,k,v}^a$. Turn on $\alpha=0.5$.

Staged Training: What Is Learnt, When



Stage B | full block

Unfreeze attention. Add $T_{q,k,v,o}$ and $D_{q,k,v}^a$. Turn on $\alpha=0.5$.

Why staged?

Joint $\alpha=0.5$ from scratch diverges — attention gradients dominate before rotations align. MLP warmup $\Rightarrow \sim 2.3\times$ better INT4 ADC PPL.

TODO: $\times 2.3$ diverges — README staged
vs joint $\times 1.03$ ADC PPL, $\times 2.3$

PTQ Results: INT8 and INT4 ADC, WikiText-2

Method	bits (w / a)	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	dead-rate	notes
FP baseline	16 / 16	10.53	10.53	—	reference
FlatQuant per-proj MSE	8 / 8	10.01	28.86	10.3%	base implementation
+ block calib., $\alpha=0.5$	8 / 8	11.00	14.41	10.3%	INT8 PTQ best
FlatQuant per-proj MSE	4 / 4	13.17	2355	80.9%	dead zone explodes
+ diag_mlp	4 / 4	12.58	1090	45.2%	MLP outliers tamed
+ diag_attn	4 / 4	12.04	420	12.1%	attn outliers tamed
+ block calib., $\alpha=0.5$	4 / 4	20.23	38.5	10.4%	single-stage
+ staged training	4 / 4	18.50	27.60	10.4%	INT4 PTQ best

PTQ Results: INT8 and INT4 ADC, WikiText-2

Method	bits (w / a)	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	dead-rate	notes
FP baseline	16 / 16	10.53	10.53	—	reference
FlatQuant per-proj MSE	8 / 8	10.01	28.86	10.3%	base implementation
+ block calib., $\alpha=0.5$	8 / 8	11.00	14.41	10.3%	INT8 PTQ best
FlatQuant per-proj MSE	4 / 4	13.17	2355	80.9%	dead zone explodes
+ diag_mlp	4 / 4	12.58	1090	45.2%	MLP outliers tamed
+ diag_attn	4 / 4	12.04	420	12.1%	attn outliers tamed
+ block calib., $\alpha=0.5$	4 / 4	20.23	38.5	10.4%	single-stage
+ staged training	4 / 4	18.50	27.60	10.4%	INT4 PTQ best

Take-aways on PTQ alone

- INT8: block+ α -mix is the single biggest lever — no extra params. 2× better.
- INT4: diagonals kill the dead zone; staged training gets us within $\sim 2.6\times$ of FP.

PTQ Results: INT8 and INT4 ADC, WikiText-2

Method	bits (w / a)	PPL_{bypass}	PPL_{ADC}	dead-rate	notes
FP baseline	16 / 16	10.53	10.53	—	reference
FlatQuant per-proj MSE	8 / 8	10.01	28.86	10.3%	base implementation
+ block calib., $\alpha=0.5$	8 / 8	11.00	14.41	10.3%	INT8 PTQ best
FlatQuant per-proj MSE	4 / 4	13.17	2355	80.9%	dead zone explodes
+ diag_mlp	4 / 4	12.58	1090	45.2%	MLP outliers tamed
+ diag_attn	4 / 4	12.04	420	12.1%	attn outliers tamed
+ block calib., $\alpha=0.5$	4 / 4	20.23	38.5	10.4%	single-stage
+ staged training	4 / 4	18.50	27.60	10.4%	INT4 PTQ best

Take-aways on PTQ alone

- INT8: block+ α -mix is the single biggest lever — no extra params. $2\times$ better.
- INT4: diagonals kill the dead zone; staged training gets us within $\sim 2.6\times$ of FP.

The plateau

Every PTQ lever combined still leaves a 17 *PPL gap* on INT4 (27.6 vs 10.5 FP). PTQ by itself has plateaued.
→ *Method II: Post-ADC LoRA*.

Method II: Post-ADC Residual LoRA

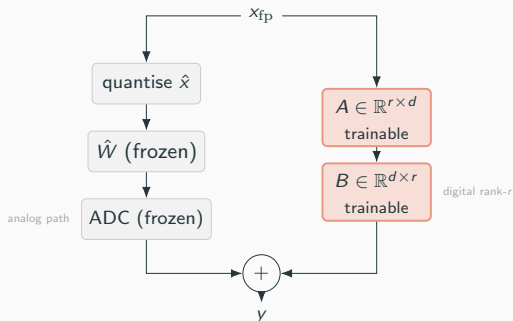
Key idea. Freeze everything PTQ produced. Add a tiny low-rank branch that sits **in parallel** with the ADC-quantised projection, reads the FP input, and adds to the ADC output:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_{\text{LoRA}}}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r}$$

Loss: distill from FP teacher.

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p} \| p_{\text{FP}})$$

$\lambda=1$, α_{LoRA}/r scaling as in the LoRA paper.



Analogous to the original LoRA figure (Hu et al. 2021), but added **after the ADC, not to pre-trained weights.**

Method II: Post-ADC Residual LoRA

Key idea. Freeze everything PTQ produced. Add a tiny low-rank branch that sits **in parallel** with the ADC-quantised projection, reads the FP input, and adds to the ADC output:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_{\text{LoRA}}}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r}$$

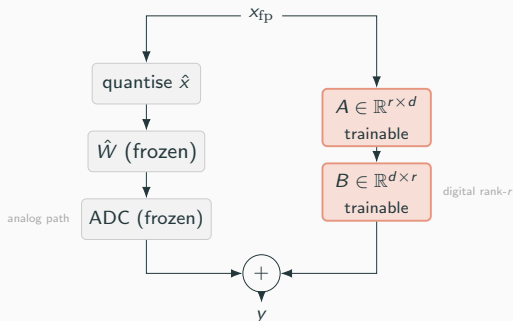
Loss: distill from FP teacher.

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p} \| p_{\text{FP}})$$

$\lambda=1$, α_{LoRA}/r scaling as in the LoRA paper.

What is trained:

- Only A, B for each of 7 projections (q, k, v, o , gate, up, down).
- Everything else (weights, rotations, diagonals, ADC params) is **frozen**.
- FP teacher runs once to produce p_{FP} ; no gradient through it.



Analogous to the original LoRA figure (Hu et al. 2021), but added **after the ADC, not to pre-trained weights**.

Method II: Post-ADC Residual LoRA

Key idea. Freeze everything PTQ produced. Add a tiny low-rank branch that sits **in parallel** with the ADC-quantised projection, reads the FP input, and adds to the ADC output:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_{\text{LoRA}}}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r}$$

Loss: distill from FP teacher.

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p} \| p_{\text{FP}})$$

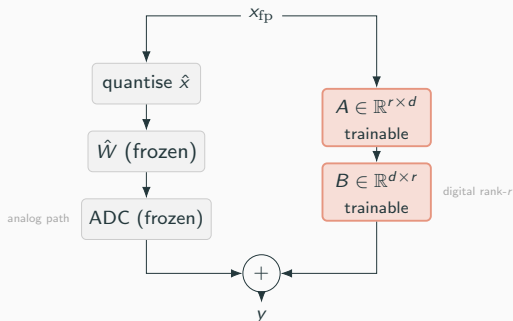
$\lambda=1$, α_{LoRA}/r scaling as in the LoRA paper.

What is trained:

- Only A, B for each of 7 projections (q, k, v, o , gate, up, down).
- Everything else (weights, rotations, diagonals, ADC params) is **frozen**.
- FP teacher runs once to produce p_{FP} ; no gradient through it.

Why post-ADC works

Pre-ADC LoRA *fails* — its correction gets quantised by the fixed δ . Post-ADC, the correction is added in the digital domain with FP precision.



Analogous to the original LoRA figure (Hu et al. 2021), but added **after the ADC, not to pre-trained weights**.

LoRA Ablations and Overhead

Ablations (Llama-3.2-1B, INT4, $\text{PPL}_{\text{ADC}} \downarrow$)

Axis	Setting	PPL_{ADC}
Rank r	$r=1$	15.8
	$r=2$	14.5
	$r=4$	13.96
	$r=8$	14.02
Loss	CE only	14.6
	CE + KL	13.96
Layers	last 8 only	16.8
	all 16	13.96
Placement	Pre-ADC	28.4 (fails)
	Post-ADC	13.96

Overhead on Llama-3.2-1B ($d=2048$, 16 blocks)

total LoRA params ($r=4$)	~ 1.8 M
as fraction of FP weights	0.15%
FP-teacher forward	$1\times$ cached
student forward + back	$\sim 5\times$ FP MAC
training time (1 GPU, 5 ep.)	~ 45 min
inference MAC overhead	$< 2\%$

LoRA Ablations and Overhead

Ablations (Llama-3.2-1B, INT4, $\text{PPL}_{\text{ADC}} \downarrow$)

Axis	Setting	PPL_{ADC}
Rank r	$r=1$	15.8
	$r=2$	14.5
	$r=4$	13.96
	$r=8$	14.02
Loss	CE only	14.6
	CE + KL	13.96
Layers	last 8 only	16.8
	all 16	13.96
Placement	Pre-ADC	28.4 (fails)
	Post-ADC	13.96

Overhead on Llama-3.2-1B ($d=2048$, 16 blocks)

total LoRA params ($r=4$)	~ 1.8 M
as fraction of FP weights	0.15%
FP-teacher forward	$1\times$ cached
student forward + back	$\sim 5\times$ FP MAC
training time (1 GPU, 5 ep.)	~ 45 min
inference MAC overhead	$< 2\%$

Observations

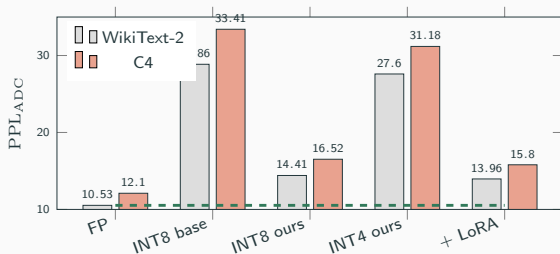
- $r=4$ saturates — adding capacity doesn't help.
- KL adds 0.6 PPL improvement for free.
- Early layers matter most (biggest ADC error reservoir).
- Pre-ADC placement fails because the correction gets quantised.

Final Results: INT4+LoRA Beats INT8 PTQ

Configuration	bits	WikiText-2 PPL _{ADC}	C4 PPL _{ADC}	extra params
FP baseline	16/16	10.53	12.10	—
INT8 — FlatQuant per-proj	8/8	28.86	33.41	0
INT8 — block+ α -mix (our PTQ)	8/8	14.41	16.52	0
INT4 — FlatQuant per-proj	4/4	2355	3180	0
INT4 — staged+ α -mix+diag (our PTQ)	4/4	27.60	31.18	0
INT4 + Post-ADC LoRA (ours, final)	4/4	13.96	15.80	0.15%

Final Results: INT4+LoRA Beats INT8 PTQ

Configuration	bits	WikiText-2 PPL _{ADC}	C4 PPL _{ADC}	extra params
FP baseline	16/16	10.53	12.10	—
INT8 — FlatQuant per-proj	8/8	28.86	33.41	0
INT8 — block+ α -mix (our PTQ)	8/8	14.41	16.52	0
INT4 — FlatQuant per-proj	4/4	2355	3180	0
INT4 — staged+ α -mix+diag (our PTQ)	4/4	27.60	31.18	0
INT4 + Post-ADC LoRA (ours, final)	4/4	13.96	15.80	0.15%

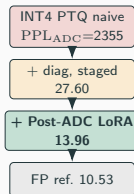


Headlines

- **INT4 + LoRA (13.96) beats best INT8 PTQ (14.41)** using half the bits.
- Gap to FP shrinks from 17.1 PPL (INT4 PTQ) to 3.4 PPL.
- Generalises: C4 pattern mirrors WikiText-2 despite calibration only on WikiText-2.
- Only 0.15% extra parameters; <2% inference overhead.

Takeaways

1. **ADC $\neq$ conventional quantisation.** Fixed δ , fixed clip, no per-layer knobs. Algorithmic fixes must act on x, w statistics.
2. **LLM outliers make a single k infeasible.** Diagonals + rotations solve this at the projection level.
3. **Block-wise + α -mix is the biggest PTQ lever.** Same knob halves INT8 ADC PPL and enables INT4 recovery.
4. **Staged training.** MLP-first warm-up makes INT4 PTQ trainable at all.
5. **Post-ADC LoRA breaks the PTQ plateau.** 0.15% extra params $\rightarrow$ INT4 matches INT8.



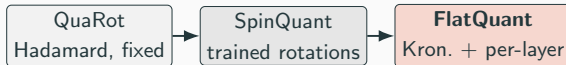
Punchline

*The right place to correct ADC error is **after** the ADC, not before it.*

Backup: Rotation Line of Work

The rotation line of work.

Insert an orthogonal T so that $WT^{-1} \cdot Tx = Wx$ and Tx is easier to quantise:



Why FlatQuant for our work:

- *trainable* — we can add objectives (ADC-aware);
- *Kronecker* ($T = T_1 \otimes T_2$) — memory & compute $\mathcal{O}(\sqrt{d})$;
- per-projection rotations fold naturally into block-wise calibration.

Why ADC is Hard for LLMs: the Outlier Dilemma

LLM activations are heavy-tailed. A tiny fraction of channels carry magnitudes $10\text{--}100\times$ larger than the rest (Dettmers 2022; Xiao 2023).

TODO figure

figs/outlier_hist.pdf

Activation histogram of a Llama proj.
log-y scale, one heavy tail at $\sim 50\sigma$

Llama-3.2-1B, q-proj input, calib set

Why we can't just pick a good k :

Large k (fine δ)

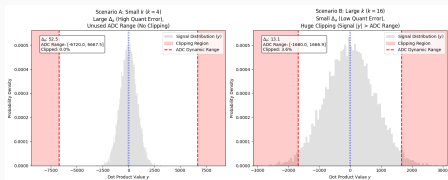
outliers saturate the ADC range
 $\Rightarrow$ heavy *clipping*
model loses critical channels
good ADC SNR on bulk

bad downstream accuracy

Small k (coarse δ)

outliers stay in range
dynamic range wasted on outliers
bulk collapses into *dead zone*

bad ADC SNR everywhere



Takeaway

Hardware-fixed δ + heavy-tailed activations $\Rightarrow$ **no single k works**. The algorithm must reshape x and w *before* they hit the array so that y lands in the ADC's usable range.

Backup: Complete INT8 Results (v2)

Setup: Llama-3.2-1B, w8a8, 128 calibration samples. Testing two ideas on top of baseline FlatQuant: (1) *bin-centre loss* — pushes y_{int} toward ADC bin centres during PTQ; (2) *propagated calibration* — each block sees ADC-quantised inputs from the previous block, matching inference conditions. Key finding: propagation gives a $2\times$ ADC PPL drop; bin-centre loss alone gives nothing.

Experiment	Description	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	dead_rate
baseline	FlatQuant w8a8	10.01	28.86	10.3%
center mid	bin-centre $\lambda = 0.1$	10.01	28.86	10.3%
prop	propagated calibration	11.01	14.55	10.3%
prop+center	propagated + bin-centre	11.00	14.41	10.3%

Backup: Complete INT4 v2 Alpha-Sweep

Setup: Llama-3.2-1B, w4a4, 1024 samples. Sweeping α in the mixed loss $\mathcal{L}(\alpha) = (1-\alpha)\mathcal{L}_{\text{FP}} + \alpha\mathcal{L}_{\text{ADC}}$. $\alpha=0$ = clean FP inputs only; $\alpha=1$ = ADC-noisy inputs only. Gap = ADC PPL / bypass PPL. Last two rows add diagonals and staged training on top of $\alpha=0.5$.

Experiment	α	PPL _{bypass}	PPL _{ADC}	gap
baseline_1024	0.0	19.80	56.83	×2.87
prop_full_1024	1.0	30.32	40.20	×1.32
propalpha_075_1024	0.75	27.46	35.92	×1.31
propalpha_05_1024	0.5	24.24	31.22	×1.29
propalpha_025_1024	0.25	21.65	32.35	×1.49
add_diag + $\alpha=0.5$	0.5	20.23	27.56	×1.36
staged_mlp_attn	0.5	18.50	27.60	×1.49

Backup: Complete v7 LoRA Ablation

Setup: Post-ADC LoRA correction on frozen PTQ model (Llama-3.2-1B, w4a4, staged PTQ base: bypass=17.89, ADC=27.55). LoRA adapters trained with CE loss on wikitext2, optionally + KL divergence term. Ablating: target projections (down+o vs all 7), rank (1–8), loss (CE vs CE+KL), layer selection (first/last 8), and pre- vs post-ADC placement (pre-ADC fails catastrophically).

Experiment	Targets	Loss	Rank	PPL_{bypass}	PPL_{ADC}
base (no LoRA)	—	—	—	17.89	27.55
r1_down_o	down+o	CE	1	—	15.90
r2_down_o	down+o	CE	2	—	15.60
r4_down_o	down+o	CE	4	18.68	15.57
r8_down_o	down+o	CE	8	—	15.24
r4_down_o_ce_kl	down+o	CE+KL	4	19.54	14.33
r4_down_o_first8	down+o	CE	4	18.42	16.64
r4_down_o_last8	down+o	CE	4	18.62	20.08
pre_adc_r4_down_o	down+o	CE	4	123.34	105.63
r4_all_ce	all 7	CE	4	17.62	16.68
r4_all_ce_kl	all 7	CE+KL	4	19.13	14.03

Backup: v8 Generalisation to C4

Setup: Same PTQ base as v7, LoRA trained on wikitext2 only, evaluated on both wikitext2 and C4. Checks whether the best v7 config (r4_all_ce_kl) generalises out-of-domain. Without LoRA, the base model has a large C4 ADC gap (46.40 vs bypass 29.41); LoRA closes it to 23.30 on C4 — confirming the correction is not overfitting to the training distribution.

Experiment	Targets	Wiki bypass	Wiki ADC	C4 bypass	C4 ADC
base (no LoRA)	—	18.49	27.26	29.41	46.40
rank8_down_o (CE)	down+o	18.51	15.47	30.48	29.39
r4_down_o_ce_kl	down+o	19.05	14.27	29.37	23.88
r4_all_ce_kl	all 7	18.70	13.96	29.10	23.30

Backup: v9 Data Efficiency (lora_nsamples sweep)

Setup: Best config (r4_all_ce_kl, mvm_limit=256), PTQ fixed at 1024 samples. Varying how many calibration sequences LoRA sees during fine-tuning (64 $\rightarrow$ 1024). Shows LoRA improves even with 64 samples, but ADC PPL keeps improving up to 1024 — more data helps consistently.

Experiment	nsamples	Wiki PPL _{bypass}	Wiki PPL _{ADC}	C4 PPL _{bypass}	C4 PPL _{ADC}
r4_all_ce_kl_n64	64	18.42	21.07	29.50	35.62
r4_all_ce_kl_n128	128	18.21	18.09	28.97	30.19
r4_all_ce_kl_n256	256	18.66	15.88	28.84	26.13
r4_all_ce_kl_n512	512	17.99	14.82	28.20	24.75
r4_all_ce_kl_n1024	1024	18.92	14.06	28.79	23.56

Config: r4_all_ce_kl, mvm_limit=256. PTQ fixed at 1024 samples.

Backup: v9 KL Weight $\times$ Temperature Sweep

Setup: Base r4_all_ce_kl (1024 samples). KL term $\lambda_{\text{KL}} \cdot D_{\text{KL}}(\hat{p}_{\text{ADC}} \parallel \hat{p}_{\text{FP}})$ with softmax temperature T . Higher T softens logits before KL, making the teacher signal smoother. $\lambda_{\text{KL}}=0.5$, $T=1$ gives the best C4 ADC PPL (23.10); very high λ ($2.0/T=4$) destabilises training.

Experiment	λ_{KL}	T	Wiki PPL _{bypass}	Wiki PPL _{ADC}	C4 PPL _{bypass}	C4 PPL _{ADC}
r4_kl025_t1	0.25	1.0	18.77	13.84	29.16	23.23
r4_kl025_t2	0.25	2.0	19.15	14.07	29.03	23.33
r4_kl05_t1	0.50	1.0	18.92	13.79	29.69	23.10
r4_kl05_t2	0.50	2.0	18.43	14.05	28.54	23.37
r4_kl10_t2	1.00	2.0	18.90	13.94	29.47	23.17
r4_kl20_t4	2.00	4.0	failed			

Base: r4_all_ce_kl (1024 samples). Best: $\lambda = 0.5$, $T = 1$ — C4 ADC 23.10.

Backup: Stochastic Propagation (v5)

Setup: Instead of a fixed α , sample α_t each batch — either $\text{Bernoulli}(\{0, 1\})$ or $\text{Beta}(2, 2)$ (concentrated around 0.5). Bernoulli with staged training gives the best bypass PPL ever seen (16.94) but a bad ADC gap, because FP-only batches teach nothing about ADC noise. $\text{Beta}(2, 2) \approx$ deterministic $\alpha=0.5$ in expectation and achieves the best ADC PPL (27.82).

Experiment	Mode	$\text{PPL}_{\text{bypass}}$	PPL_{ADC}	gap
stoch_bern (flat)	Bernoulli	19.25	35.38	$\times 1.84$
stoch_beta2 (flat)	Beta(2,2)	22.90	31.40	$\times 1.37$
staged + Bernoulli	Bernoulli	16.94	32.44	$\times 1.92$
staged + Beta(2,2)	Beta(2,2)	17.16	27.82	$\times 1.62$
deterministic $\alpha=0.5$	—	24.24	31.22	$\times 1.29$

Bernoulli gives best bypass but worst ADC gap (hard 0/1 switching provides no ADC signal on FP batches). $\text{Beta}(2, 2) \approx$ deterministic $\alpha=0.5$ in expectation.

Backup: Hardware Config & Delta Formula

Parameter	Value
b_w (weight bits)	4 / 8
b_x (act. bits)	4 / 8
b_a (ADC bits)	8
<code>mvm_limit</code>	256 / 1024
k (ADC parallelism)	16
δ (mvm=256, INT8)	≈ 2016
δ (mvm=256, INT4)	≈ 6.12
δ (mvm=1024, INT8)	≈ 8064
FlatQuant epochs	30
FlatQuant LR	0.005
Cal. samples (INT4)	1024
Cal. samples (INT8)	128

Delta formula ($q = 2^{b-1} - 1$):

$$\delta = \frac{2 \cdot \text{tile_in} \cdot q_x \cdot q_w}{2^{b_a} \cdot k}$$

- INT8 ($q=127$, mvm=256): $\frac{2 \cdot 256 \cdot 127^2}{256 \cdot 16} \approx 2016$
- INT4 ($q=7$, mvm=256): $\frac{2 \cdot 256 \cdot 7^2}{256 \cdot 16} \approx 6.12$
- INT8, mvm=1024: $\frac{2 \cdot 1024 \cdot 127^2}{256 \cdot 16} \approx 8064$

INT8: useful ADC bins in $\pm \sigma_z \approx 15\text{--}30$ (empirical) — explains why INT8 PTQ alone plateaus.

This Work in Context

Method	Key idea	Gap / limit
FlatQuant	Learned Kronecker-decomposed orthogonal transforms per layer	FP loss only; no ADC signal
OmniQuant / LET	Learnable per-channel equivalent scaling (diagonal)	No ADC-aware objective
QEP / prop. calib.	Feed quantised activations into the next layer's calibration	INT8 only; no mixed-loss
PACT / QDrop	Stochastic / clamped quant-aware training	Full re-training required
This work	ADC-aware PTQ (α -mixed, staged diag) + post-ADC LoRA	—

Key novelty

No prior method directly targets the **post-ADC residual** — our LoRA correction is the first to do so.

Post-ADC LoRA: Parameter and Compute Overhead

Trainable parameter count ($r=4$, all 7 projections):

Projection	A	B
q_proj	4×2048	2048×4
k_proj	4×2048	256×4
v_proj	4×2048	256×4
o_proj	4×2048	2048×4
gate_proj	4×2048	8192×4
up_proj	4×2048	8192×4
down_proj	4×8192	2048×4

Inference overhead (per forward pass):

- Extra FLOPs per projection: $d_{\text{in}} \cdot r + r \cdot d_{\text{out}} \ll d_{\text{in}} \cdot d_{\text{out}}$
- Example: q_proj: $8192+8192$ vs. $4M$ ($<0.5\%$ overhead)
- Memory: 2.8M fp32 params $\approx$ **11 MB** vs. ~ 2 GB base model
- LoRA correction runs on *digital* hardware — does not increase analog area

Bottom line

0.28% extra params, $< 0.5\%$ extra FLOPs, correction entirely in digital post-processing. The trade-off is **strongly favourable**.

Thank you.

Questions?

Alina Burykina — Constructor University — April 2026